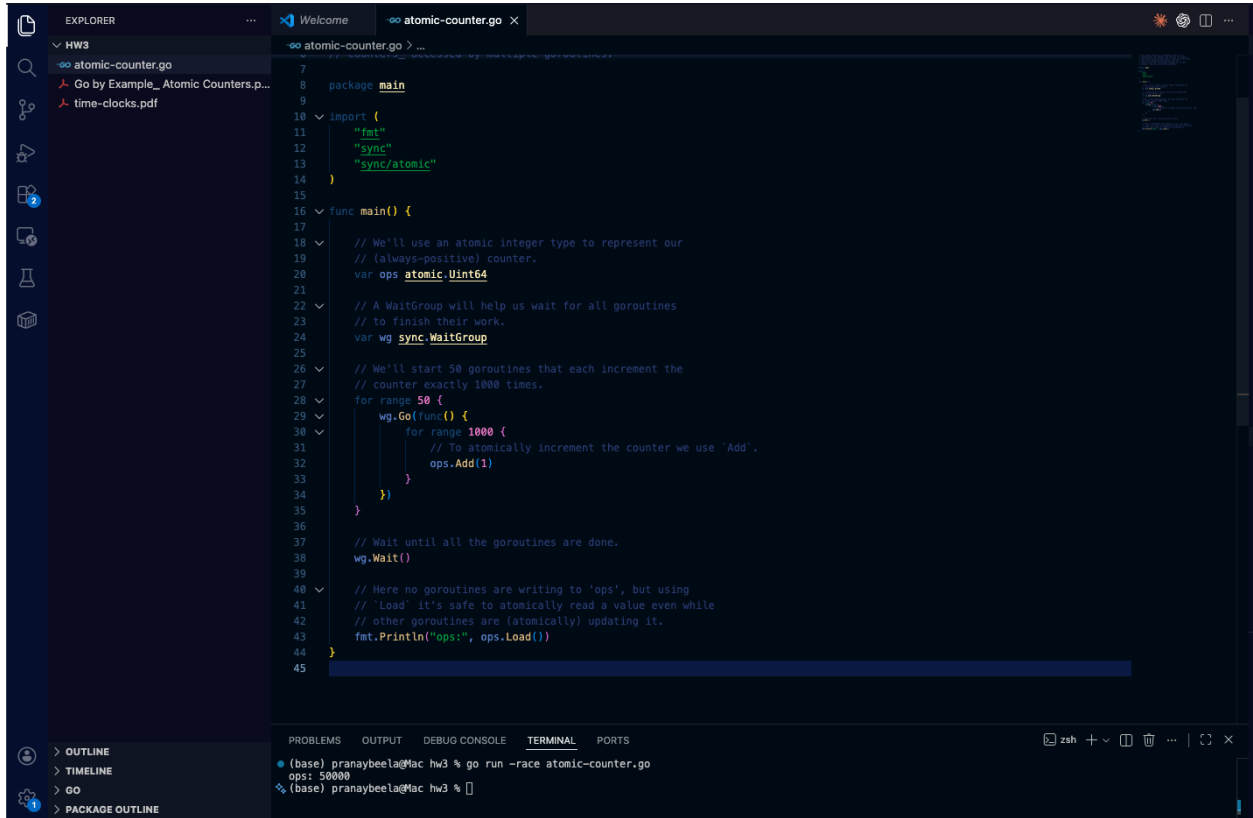


Atomicity

With `atomic` (`ops.Add(1)`): we will always see the result to be 50000 because each increment is thread-safe/atomic.

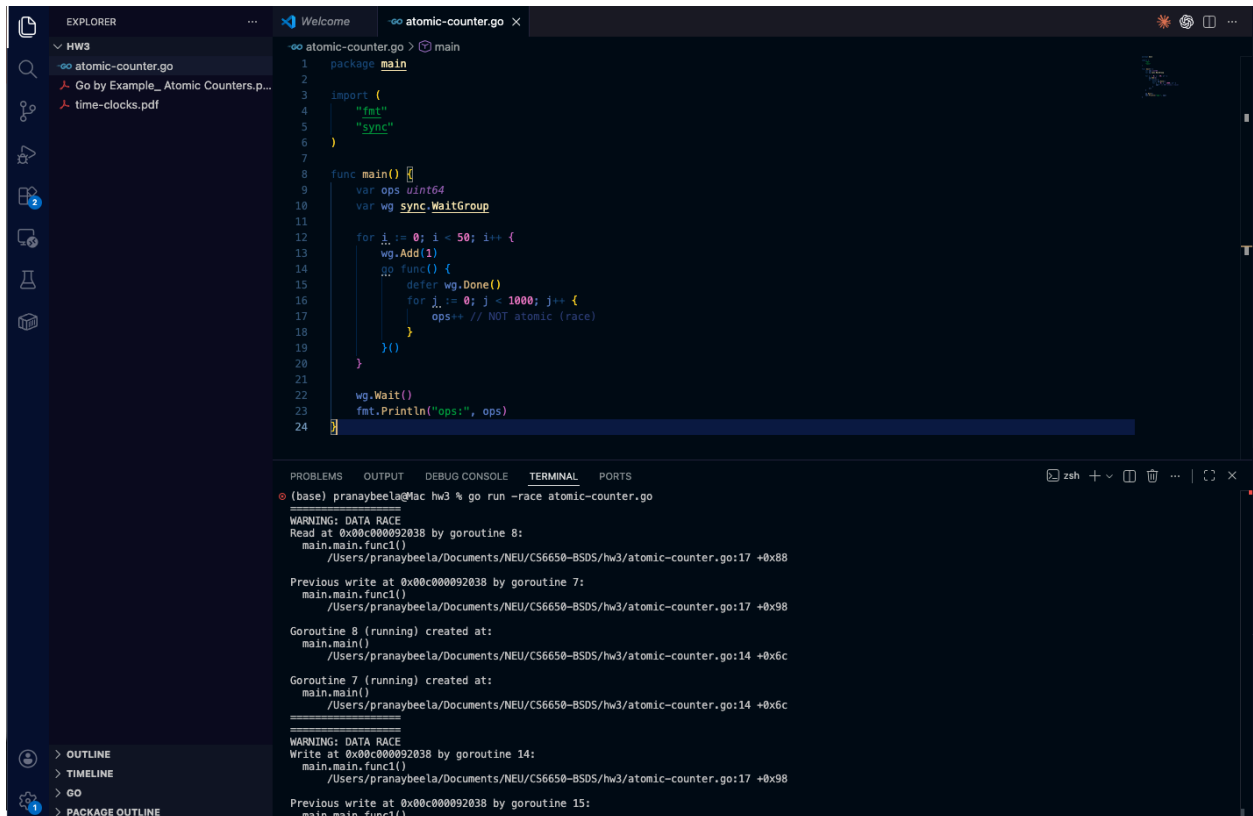


```
7
8 package main
9
10 import (
11     "fmt"
12     "sync"
13     "sync/atomic"
14 )
15
16 func main() {
17     // We'll use an atomic integer type to represent our
18     // (always-positive) counter.
19     var ops atomic.Uint64
20
21     // A WaitGroup will help us wait for all goroutines
22     // to finish their work.
23     var wg sync.WaitGroup
24
25     // We'll start 50 goroutines that each increment the
26     // counter exactly 1000 times.
27     for range 50 {
28         wg.Go(func() {
29             for range 1000 {
30                 // To atomically increment the counter we use 'Add'.
31                 ops.Add(1)
32             }
33         })
34     }
35
36     // Wait until all the goroutines are done.
37     wg.Wait()
38
39     // Here no goroutines are writing to 'ops', but using
40     // 'Load' it's safe to atomically read a value even while
41     // other goroutines are (atomically) updating it.
42     fmt.Println("ops:", ops.Load())
43 }
44
45
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
• (base) pranaybeela@Mac hw3 % go run -race atomic-counter.go
ops: 50000
(base) pranaybeela@Mac hw3 %
```

Without atomic (ops++): we will often see a different number (usually < 50000) and -race prints DATA RACE because multiple goroutines are reading/writing ops at the same time which implies that there would be updates lost.



The screenshot shows a VS Code editor with a Go file named `atomic-counter.go`. The code defines a `main` package with `fmt` and `sync` imports. The `main` function declares `ops uint64` and a `sync.WaitGroup`. It then runs a loop where `i` goes from 0 to 50, and for each `i`, it calls `wg.Add(1)` and launches a goroutine `func()`. Inside the goroutine, it calls `defer wg.Done()` and runs a loop where `j` goes from 0 to 1000, incrementing `ops++` (commented as `// NOT atomic (race)`). After the goroutines finish, it calls `wg.Wait()` and prints `ops`.

The terminal output shows the command `go run -race atomic-counter.go` being executed. It displays several warnings for data races, including a read by goroutine 8 and writes by goroutines 7, 14, and 15, all occurring at memory address `0x00c000092038`.

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
6 )
7
8 func main() {
9     var ops uint64
10    var wg sync.WaitGroup
11
12    for i := 0; i < 50; i++ {
13        wg.Add(1)
14        go func() {
15            defer wg.Done()
16            for j := 0; j < 1000; j++ {
17                ops++ // NOT atomic (race)
18            }
19        }()
20    }
21
22    wg.Wait()
23    fmt.Println("ops:", ops)
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) pranaybeela@Mac hw3 % go run -race atomic-counter.go

WARNING: DATA RACE
Read at 0x00c000092038 by goroutine 8:
main.main.func1()
/Users/pranaybeela/Documents/NEU/CS6650-BSDS/hw3/atomic-counter.go:17 +0x88

Previous write at 0x00c000092038 by goroutine 7:
main.main.func1()
/Users/pranaybeela/Documents/NEU/CS6650-BSDS/hw3/atomic-counter.go:17 +0x98

Goroutine 8 (running) created at:
main.main()
/Users/pranaybeela/Documents/NEU/CS6650-BSDS/hw3/atomic-counter.go:14 +0x6c

Goroutine 7 (running) created at:
main.main()
/Users/pranaybeela/Documents/NEU/CS6650-BSDS/hw3/atomic-counter.go:14 +0x6c

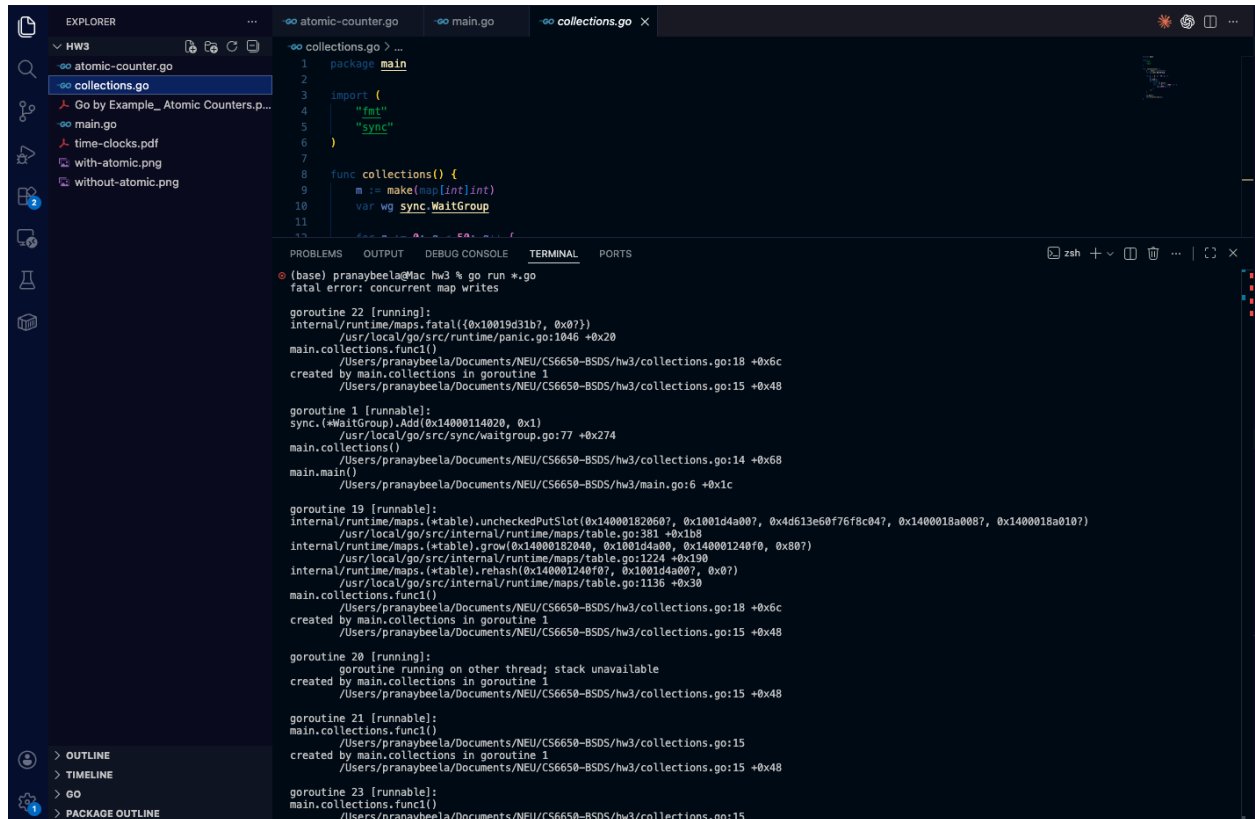
=====

WARNING: DATA RACE
Write at 0x00c000092038 by goroutine 14:
main.main.func1()
/Users/pranaybeela/Documents/NEU/CS6650-BSDS/hw3/atomic-counter.go:17 +0x98

Previous write at 0x00c000092038 by goroutine 15:
main.main.func1()

Collections

Without Mutex: Concurrent map writes means multiple goroutines are writing to the same Go map at the same time. Plain map is not thread-safe, so Go crashes to prevent corruption even if the keys are different.



The screenshot shows an IDE with a Go file named `collections.go` and a terminal window. The code in `collections.go` is as follows:

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
6 )
7
8 func collections() {
9     m := make(map[int]int)
10    var wg sync.WaitGroup
11
12    for i := 0; i < 100000; i++ {
13        go func(i int) {
14            m[i] = i
15        }(i)
16    }
17    wg.Wait()
18 }
```

The terminal output shows the following error and stack trace:

```
(base) pranybeela@Mac hw3 % go run *.go
fatal error: concurrent map writes

goroutine 22 [running]:
internal/runtime/maps.fatal({0x10019d31b?, 0x0?})
    /usr/local/go/src/runtime/panic.go:1046 +0x20
main.collections.func1()
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:18 +0x6c
created by main.collections in goroutine 1
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:15 +0x48

goroutine 1 [runnable]:
sync.(*WaitGroup).Add(0x14000114020, 0x1)
    /usr/local/go/src/sync/waitgroup.go:77 +0x274
main.collections()
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:14 +0x68
main.main()
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/main.go:6 +0x1c

goroutine 19 [runnable]:
internal/runtime/maps.(*table).uncheckedPutSlot(0x14000102060?, 0x1001d4a00?, 0x4d613e60f76f8c047, 0x1400018a008?, 0x1400018a010?)
    /usr/local/go/src/internal/runtime/maps/table.go:381 +0x1b8
internal/runtime/maps.(*table).grow(0x14000102040, 0x1001d4a00, 0x140001240f0, 0x807)
    /usr/local/go/src/internal/runtime/maps/table.go:1224 +0x190
internal/runtime/maps.(*table).rehash(0x140001240f0?, 0x1001d4a00?, 0x807)
    /usr/local/go/src/internal/runtime/maps/table.go:1136 +0x30
main.collections.func1()
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:18 +0x6c
created by main.collections in goroutine 1
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:15 +0x48

goroutine 20 [running]:
goroutine running on other thread; stack unavailable
created by main.collections in goroutine 1
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:15 +0x48

goroutine 21 [runnable]:
main.collections.func1()
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:15
created by main.collections in goroutine 1
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:15 +0x48

goroutine 23 [runnable]:
main.collections.func1()
    /Users/pranybeela/Documents/NEU/CS6650-BSDS/hw3/collections.go:15
```

With Mutex: A plain Go map is not safe for concurrent access, so we must synchronize, we are doing that using **sync.Mutex** here. A mutex prevents concurrent map access, so results are correct, here, len=50000. The trade-off is it makes updates happen one at a time, reducing parallelism and slowing things down.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 12.361167ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 11.960167ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 6.178459ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 6.106292ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 6.150417ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 6.103292ms
❖ (base) pranaybeela@Mac hw3 %
```

With RWMutex:

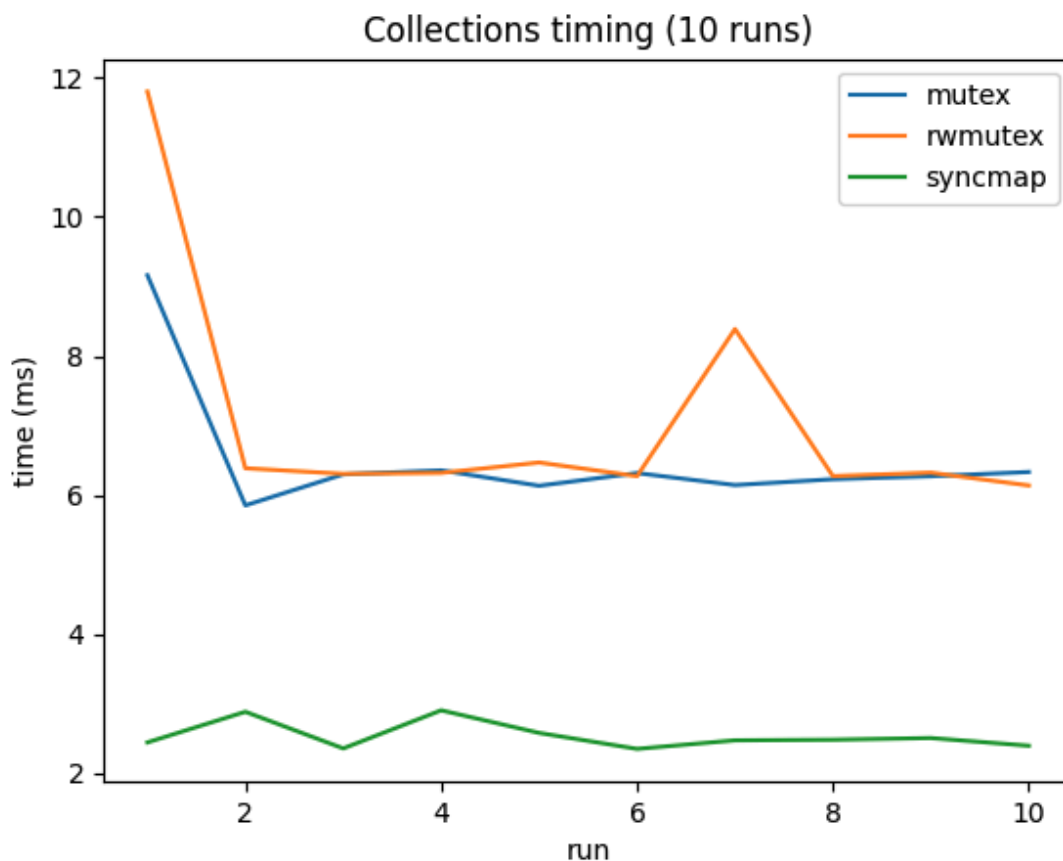
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 12.323292ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 12.503125ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 6.333291ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 6.328875ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 6.28175ms
● (base) pranaybeela@Mac hw3 % go run *.go
len: 50000
time: 6.17225ms
❖ (base) pranaybeela@Mac hw3 %
```

With Sync.Map:

```
• (base) pranaybeela@Mac hw3 % go run *.go  
len: 50000  
time: 4.035708ms  
• (base) pranaybeela@Mac hw3 % go run *.go  
len: 50000  
time: 3.331625ms  
• (base) pranaybeela@Mac hw3 % go run *.go  
len: 50000  
time: 3.573958ms  
• (base) pranaybeela@Mac hw3 % go run *.go  
len: 50000  
time: 4.040709ms  
❖ (base) pranaybeela@Mac hw3 %
```

Comparing the run times:



Why these results?

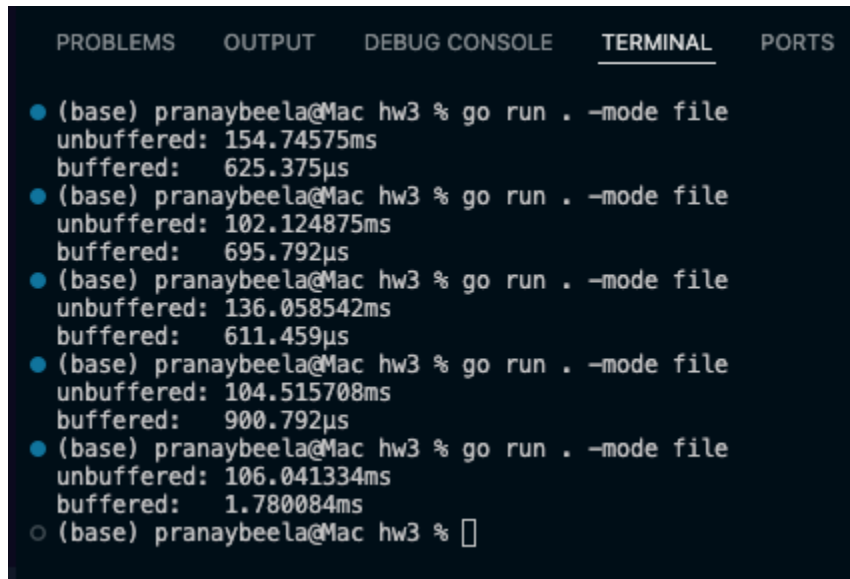
Every $m[k]=v$ mutates a shared state, so **Mutex** and **RWMutex** force one writer at a time, creating contention and similar runtimes. **sync.Map** is designed for concurrent access and reduces lock contention internally, so it's often faster here.

If reads dominate: **RWMutex** can win vs **Mutex** because many goroutines can hold **RLock** at once (parallel reads). **Mutex** still allows only one reader or writer at a time, so it bottlenecks.

Tradeoffs:

1. **Mutex**: Simplest, type-safe $map[K]V$, good general default; can bottleneck under high concurrency.
2. **RWMutex**: Best when there are a lot of reads and few writes. A write still blocks all readers and writers, and **RWMutex** adds overhead can be slower if not read-heavy.
3. **sync.Map**: Easiest safe concurrent map without manual locks. Often used strong for high concurrency and read-heavy or stable keys, but is less type-safe because it stores values as any, it uses a different API like Store, Load, and Range, and it is not guaranteed to be the fastest choice for every workload, especially when writes are very frequent.

File Access:



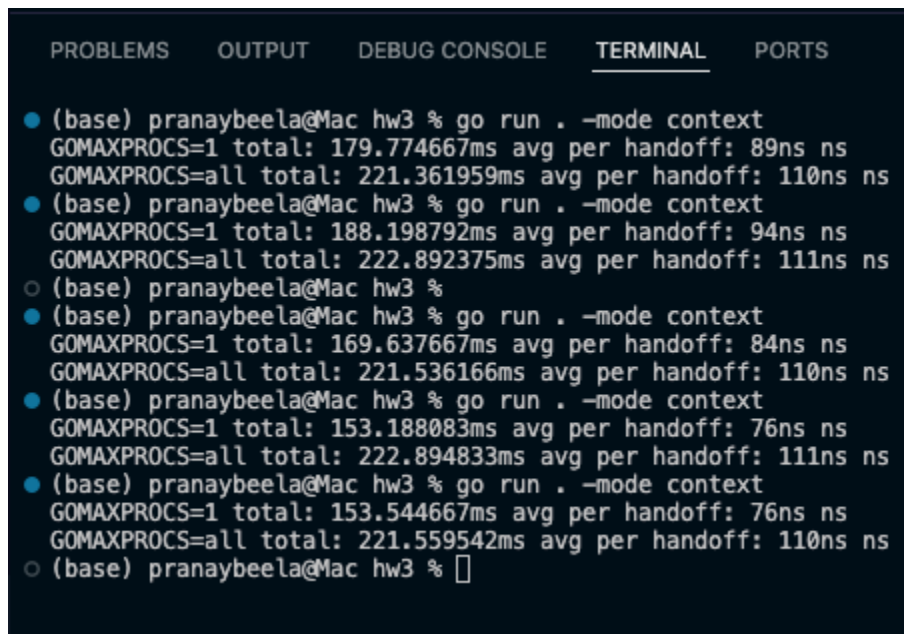
The screenshot shows a Go IDE interface with a terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal output shows six benchmark runs comparing unbuffered and buffered file access. Each run shows the command `go run . -mode file` and the resulting times in milliseconds (ms) for unbuffered and microseconds (μs) for buffered. The buffered mode is consistently faster than the unbuffered mode.

```
(base) pranaybeela@Mac hw3 % go run . -mode file
unbuffered: 154.74575ms
buffered: 625.375μs
(base) pranaybeela@Mac hw3 % go run . -mode file
unbuffered: 102.124875ms
buffered: 695.792μs
(base) pranaybeela@Mac hw3 % go run . -mode file
unbuffered: 136.058542ms
buffered: 611.459μs
(base) pranaybeela@Mac hw3 % go run . -mode file
unbuffered: 104.515708ms
buffered: 900.792μs
(base) pranaybeela@Mac hw3 % go run . -mode file
unbuffered: 106.041334ms
buffered: 1.780084ms
(base) pranaybeela@Mac hw3 %
```

Why: unbuffered does a write call per line -tons of expensive OS work. Buffered batches many lines in memory and writes in bigger chunks - far fewer OS calls.

Tradeoff: buffered needs `Flush()/Close()` to guarantee data is written; it uses extra memory and can lose the last buffered chunk if the program crashes before flushing, but it's much faster

Context Switching:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● (base) pranaybeela@Mac hw3 % go run . -mode context
GOMAXPROCS=1 total: 179.774667ms avg per handoff: 89ns ns
GOMAXPROCS=all total: 221.361959ms avg per handoff: 110ns ns
● (base) pranaybeela@Mac hw3 % go run . -mode context
GOMAXPROCS=1 total: 188.198792ms avg per handoff: 94ns ns
GOMAXPROCS=all total: 222.892375ms avg per handoff: 111ns ns
○ (base) pranaybeela@Mac hw3 %
● (base) pranaybeela@Mac hw3 % go run . -mode context
GOMAXPROCS=1 total: 169.637667ms avg per handoff: 84ns ns
GOMAXPROCS=all total: 221.536166ms avg per handoff: 110ns ns
● (base) pranaybeela@Mac hw3 % go run . -mode context
GOMAXPROCS=1 total: 153.188083ms avg per handoff: 76ns ns
GOMAXPROCS=all total: 222.894833ms avg per handoff: 111ns ns
● (base) pranaybeela@Mac hw3 % go run . -mode context
GOMAXPROCS=1 total: 153.544667ms avg per handoff: 76ns ns
GOMAXPROCS=all total: 221.559542ms avg per handoff: 110ns ns
○ (base) pranaybeela@Mac hw3 %
```

- GOMAXPROCS=1 avg handoff is ~76–94 ns (total ~153–222 ms)
- GOMAXPROCS=all avg handoff is ~110–111 ns (total ~221–223 ms)

Which is faster? GOMAXPROCS=1 is faster here.

Why? With 1 OS thread, both goroutines stay on the same thread, so the scheduler and cache behavior are simpler. With multiple threads, the goroutines can bounce across threads/cores, adding extra scheduling + cache/coherency overhead for the channel handoff.

Relation to processes/containers/VMs: goroutine context switches are the cheapest; switching between OS threads/processes is heavier, and VMs are heavier still because there's more state and isolation involved.

Locust:

 LOCUST

Host

http://localhost:8080

Status

READY

RPS

0

Failures

0%



Start new load test

Number of users (peak concurrency) *

1

Ramp up (users started/second) *

1

Host

http://localhost:8080

Advanced options

▼

START

 LOCUST

Host

http://localhost:8080

Status

RUNNING

Users

1

RPS

3.1

Failures

0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

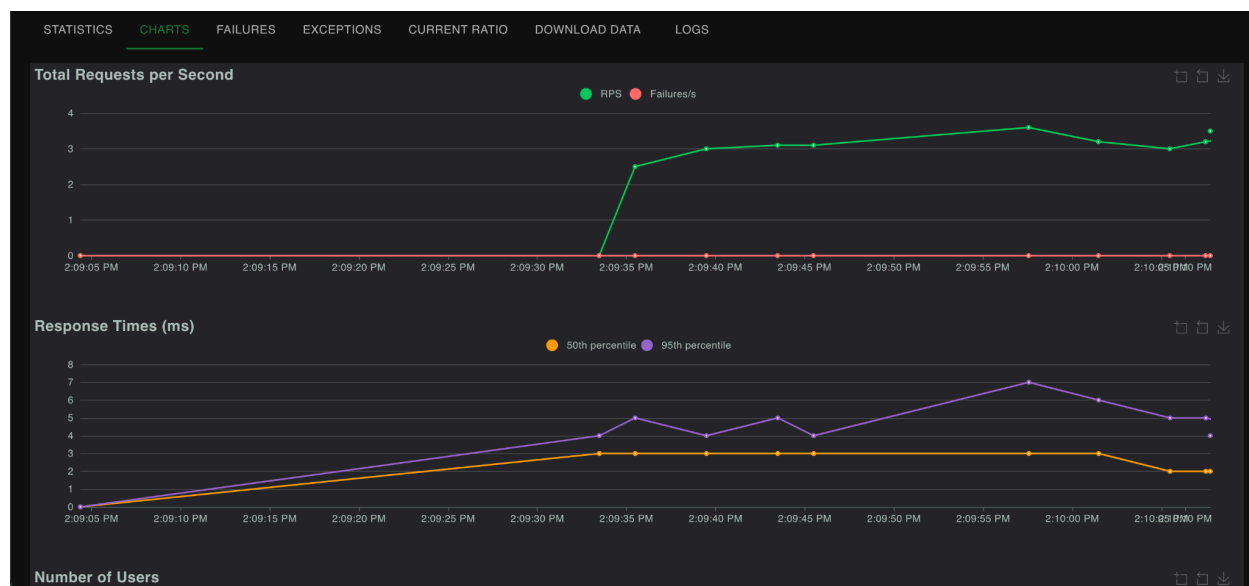
CURRENT RATIO

DOWNLOAD DATA

LOGS

⏏

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	21	0	3	4	6	2.85	1	6	753.71	1.5	0
POST	/albums	5	0	3	14	14	5.14	2	14	97.8	0.4	0
GET	/albums/1	5	0	3	3	3	2.45	2	3	95	0.3	0
GET	/albums/2	5	0	2.11	3	3	2.58	2	3	90	0.4	0
GET	/albums/3	9	0	3	49	49	7.89	1	49	117	0.5	0
Aggregated		45	0	3	6	49	4.04	1	49	406.56	3.1	0



 **LOCUST**

Host
http://localhost:8080

Status
RUNNING

Users
1

RPS
3.6

Failures
0%

EDIT

STOP

RESET

⚙️

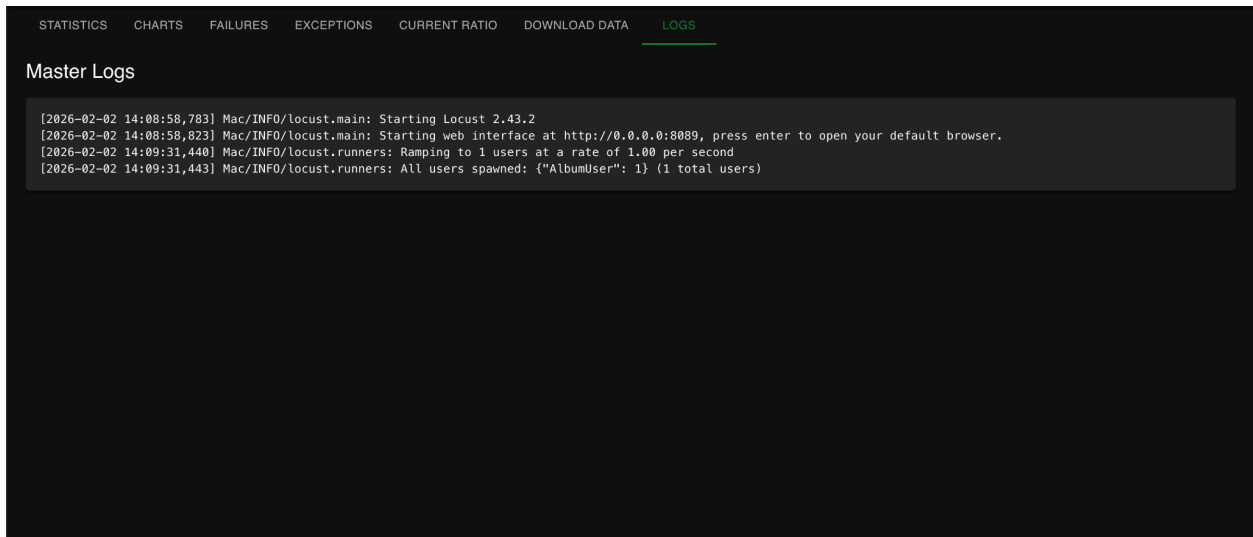
STATISTICS **CHARTS** FAILURES EXCEPTIONS **CURRENT RATIO** DOWNLOAD DATA LOGS

Ratio Per Class

- 100.0% AlbumUser
 - 33.3% getAlbumById
 - 50.0% getAlbums
 - 16.7% postAlbum

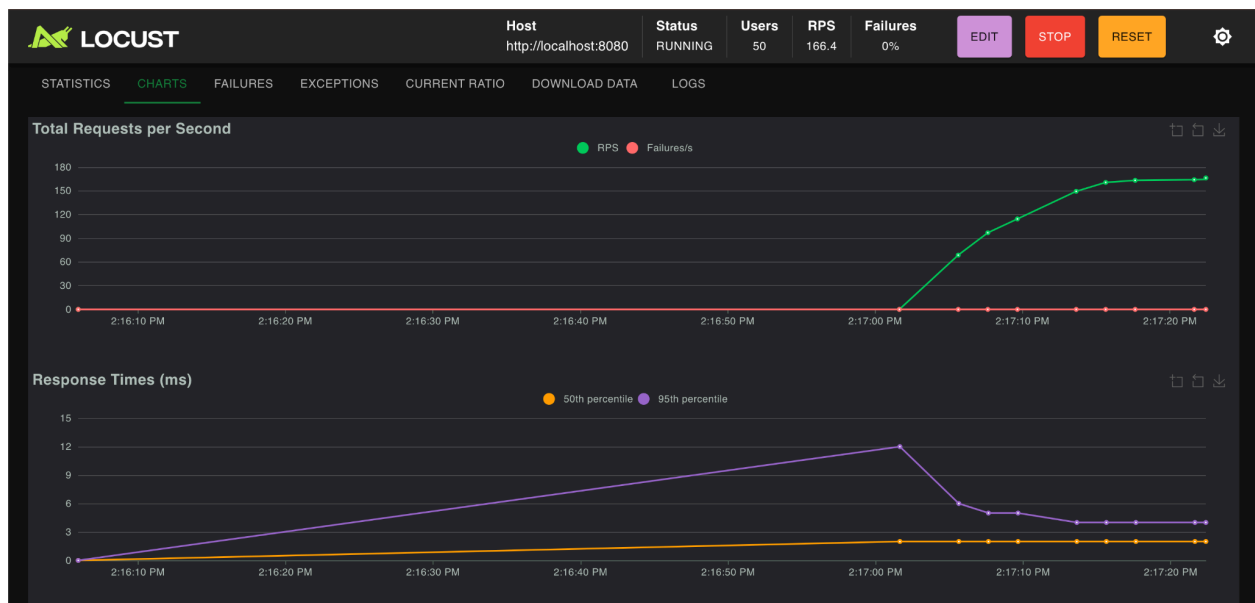
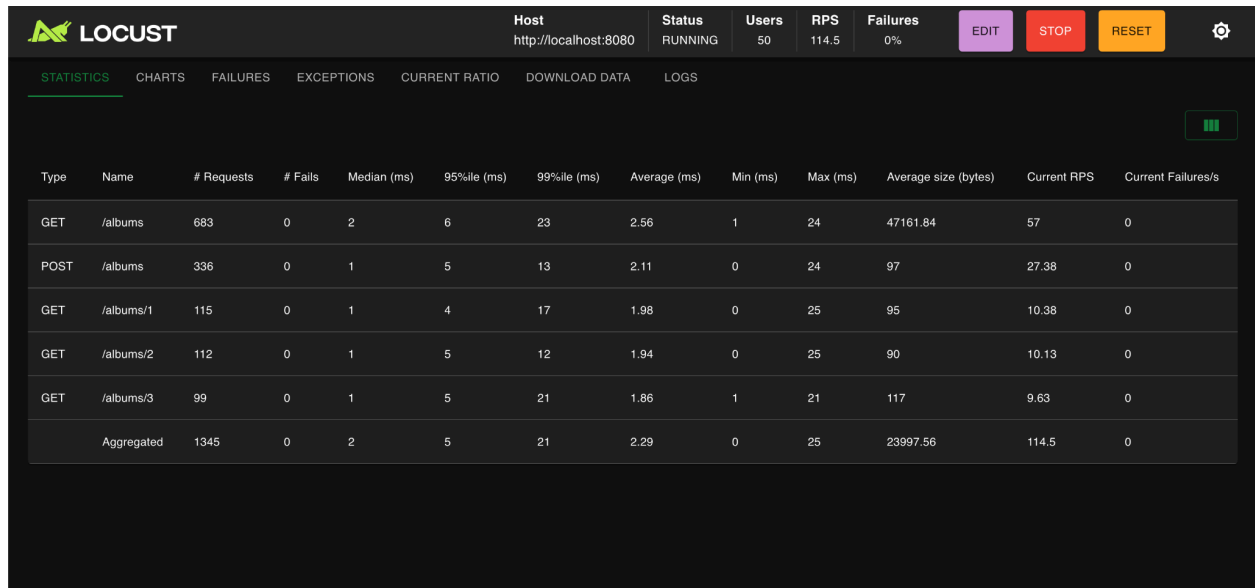
Total Ratio

- 100.0% AlbumUser
 - 33.3% getAlbumById
 - 50.0% getAlbums
 - 16.7% postAlbum



- What's going on is simply that different endpoints do different amounts of work, so their latency/RPS differ. GET /albums and GET /albums/:id are mostly read + JSON serialization, so they're usually faster and can hit higher RPS.
- **POST /albums does extra work:** parse JSON (BindJSON), validate/allocate, append to the shared slice, and return a response. That overhead makes it slower and lowers RPS.
- **Why the numbers vary even within GETs:**
 - **Payload size differs:** /albums returns the whole list, /albums/:id returns one item, different JSON size and serialization time.
 - Tail latency spikes happen from Go runtime/OS scheduling, GC pauses, and background CPU noise, so p95/p99/max can jump even with 1 user.
 - Your task weights control how often each endpoint is hit, so "Aggregated" stats reflect the mix, not a single route.

Local Test:



Amdahl's Law:

- GET vs POST throughput differences make sense because GET mostly reads + serializes JSON, while POST also parses JSON and updates shared state.
- Adding more workers/users will increase throughput, but it won't scale linearly because some parts stay serial or contended (server CPU, shared data updates, GC/runtime overhead, network/HTTP overhead).
- If multiple requests read/write the same shared structure (like a map/slice), contention/races/locking can limit scaling and increase tail latency.

Context Switching:

- FastHttpUser reduces client-side overhead compared to HttpUser, so Locust can generate higher RPS with lower latency.
- After switching, the bottleneck usually moves to the server (CPU, JSON serialization, shared-state contention), so improvements plateau once the server is saturated.